DECLARATION

We hereby declare that the Project entitled "GraphQL-Shield: Dual-Layer Enforcement Against GraphQL BOLA with Cross-Layer Coverage Evidence" to be submitted for the course **"[COURSE CODE] – Computer Communication and Network Security"**, is our original work as a team.

Place: Chennai
Date: [DD/MM/YYYY]

[Student Name 1] [Register Number]

[Student Name 2] [Register Number]

[Student Name 3] [Register Number]

[Student Name 4] [Register Number]

ABSTRACT

Broken Object Level Authorization (BOLA), also called Insecure Direct Object Reference (IDOR), is ranked first in the OWASP API Security Top 10 (API1:2023). GraphQL makes it harder to stop: one request can reach an object by identifier, through nested lookups, fragments, variables and batches, or through relationship fields that never name the object at all. This project presents GraphQL-Shield, a security gateway that enforces object ownership at two independent points. Layer 1 is a schema-aware pre-execution gate that checks every identifier it can resolve before any resolver runs. Layer 2 is an execution-time guard inside the GraphQL engine that checks every object resolvers return and every mutation target the engine coerces. Because Layer 2 records each object Layer 1 never evaluated, the gateway measures its own pre-execution coverage.

We evaluate this evidence by fault injection: 17 realistic bugs planted in Layer 1, each run on 5 seeds of 1,000 requests, were all detected with no false alarms. The same evidence drives a self-repairing configuration that restored all 10 planted configuration mistakes (30 of 30 runs) and refused all 33 wrong decoy repairs. Four security properties (safety, non-interference, write safety, availability) are checked with property-based testing, four leak channels were found and closed, and no timing difference remains between foreign and missing objects. Protection costs about 1 ms per request. Compared with the Node.js library graphql-shield on the same schema, both stop all 13 attacks, but graphql-shield loses an owner's own data to GraphQL null propagation while GraphQL-Shield does not.

Keywords: GraphQL, API security, BOLA, IDOR, access control, defence in depth, fault injection, property-based testing, XGBoost.

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to the entire management of SRM Institute of Science and Technology for providing us with the necessary facilities for the completion of this project.

We wish to express our deep sense of gratitude and sincere thanks to our Professor and Head of the Department [HoD Name], for the encouragement, timely help, and advice offered to us. We wish to express our sincere thanks to our Professor in-charge [Professor In-charge Name], for the constant motivation.

We would like to express our deepest gratitude to our guide, [Guide Name], for the valuable guidance, consistent encouragement, personal caring, timely help and for providing us with an excellent atmosphere for doing research. All through the work, in spite of a busy schedule, our guide has extended cheerful and cordial support to us for completing this research work.

We would like to express our sincere thanks to our faculty coordinator [Coordinator Name] for the time and suggestions for the implementation of this project.

We also extend our gratitude and thanks to all the teaching and non-teaching staff of the Electronics and Communication Engineering Department and to our parents and friends, who extended their kind cooperation using valuable suggestions and timely help during this project work.

Authors

([Student Name 1])

([Student Name 2])

([Student Name 3])

([Student Name 4])

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	DECLARATION	iii
	ABSTRACT	iv
	ACKNOWLEDGEMENT	v
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	ABBREVIATIONS	ix
1	**INTRODUCTION**	1
2	**PROBLEM STATEMENT & MOTIVATION**	3
3	**OBJECTIVES**	5
4	**LITERATURE SURVEY**	6
5	**RESEARCH GAP**	9
6	**PROPOSED SYSTEM**	10
7	**SYSTEM ARCHITECTURE / METHODOLOGY**	12
8	**IMPLEMENTATION**	16
9	**RESULTS AND DISCUSSION**	21
10	**CONCLUSION**	27
11	**FUTURE SCOPE**	28
	REFERENCES	29
	APPENDIX – RESEARCH PAPER TEMPLATE	31

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Forms of BOLA in GraphQL	4
4.1	Summary of Literature Survey	7
7.1	Gap Causes and the Repairs Proposed	14
8.1	Technology Stack	16
8.2	Main Source Modules	16
8.3	Account Protection	18
8.4	Main API Endpoints	19
8.5	Demonstration Scenarios (User 101 as Attacker)	19
8.6	Dashboard Tabs and What They Show	20
9.1	Layer Ablation: the Layer that Stopped Each Attack	21
9.2	Fault Injection: 17 Layer 1 Faults, 5 Seeds of 1,000 Requests Each	22
9.3	Timing Channel: Median Time of Foreign Minus Missing Object (ms)	23
9.4	Comparison with graphql-shield	24
9.5	Paired Overhead of Protection	25
9.6	Latency (ms) as a Folder Returns More Children (10% Foreign)	25
9.7	Self-Repair: 10 Configuration Mistakes, 3 Seeds of 400 Requests	25
A.1	Research Paper Details	31

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
7.1	GraphQL-Shield Architecture	12
7.2	Lifecycle of a Request through GraphQL-Shield	13
9.1	Predicting Requests Affected by Injected Layer 1 Faults	22
9.2	Mean Latency per Request (10 × 100 Interleaved Requests)	24

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
BOLA	Broken Object Level Authorization
IDOR	Insecure Direct Object Reference
OWASP	Open Worldwide Application Security Project
AST	Abstract Syntax Tree
SDL	Schema Definition Language
JWT	JSON Web Token
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
WAF	Web Application Firewall
CVE	Common Vulnerabilities and Exposures
PBKDF2	Password-Based Key Derivation Function 2
SHA	Secure Hash Algorithm
DB	Database
ORM	Object-Relational Mapping
WAL	Write-Ahead Logging
XGBoost	Extreme Gradient Boosting
SHAP	SHapley Additive exPlanations
RQ	Research Question
CI	Confidence Interval / Continuous Integration
F1	Harmonic mean of precision and recall
ABAC	Attribute-Based Access Control
TLS	Transport Layer Security
SIEM	Security Information and Event Management
L1 / L2	Layer 1 (pre-execution gate) / Layer 2 (execution-time guard)

CHAPTER 1

INTRODUCTION

Modern web and mobile applications talk to their back-ends through Application Programming Interfaces (APIs). Over the last decade **GraphQL**, created at Facebook and now maintained by the GraphQL Foundation, has become a popular alternative to REST. Instead of many fixed endpoints, a GraphQL server exposes one endpoint and a typed *schema*; the client writes a query that selects exactly the fields and nested objects it needs, and the server answers in one round trip.

This flexibility is also a security problem. In network security terms, every request that crosses the network boundary to the API is authenticated (the server knows *who* the caller is), but the server must still decide, for every object it returns or changes, whether that caller is *allowed* to see or change that particular object. When this object-level check is missing or incomplete, the API has a **Broken Object Level Authorization (BOLA)** flaw: an authenticated user can read or modify another user's data simply by asking for it.

BOLA is the number one risk in the OWASP API Security Top 10 (API1:2023) [1]. It is hard to detect with traditional network defences such as firewalls and intrusion detection systems, because a BOLA request is ordinary, well-formed, authenticated traffic; the only difference is *whose* object it names.

GraphQL-Shield is our answer to this problem. It is a security gateway built with Python, FastAPI and Strawberry/graphql-core that sits in front of a GraphQL application and enforces object ownership at two independent layers:

- **Layer 1 – Pre-execution gate:** parses the incoming request before execution and blocks it with HTTP 403 if it names any object the caller does not own.
- **Layer 2 – Execution-time guard:** runs inside the GraphQL engine, checks every object that resolvers actually return and every mutation target, removes foreign objects, and refuses unauthorised writes.

The key idea of the project is that the second layer also *audits the first*: every object Layer 2 sees that Layer 1 never checked is recorded as evidence of a Layer 1 blind spot. This evidence is used

to detect bugs, train a blind-spot predictor, and automatically propose verified configuration repairs.

The remainder of this report is organised as follows. Chapter 2 states the problem and motivation, Chapter 3 lists the objectives, Chapter 4 surveys the literature and Chapter 5 identifies the research gap. Chapter 6 presents the proposed system, Chapter 7 its architecture and methodology, and Chapter 8 the implementation. Chapter 9 reports the results, Chapter 10 concludes, and Chapter 11 outlines future scope, followed by the references and the research paper prepared from this work.

CHAPTER 2

PROBLEM STATEMENT & MOTIVATION

2.1 Problem Statement

> *To design and implement a runtime defence that prevents authenticated users of a GraphQL API from reading or modifying objects they do not own – in every form GraphQL allows such access – while keeping legitimate access intact, adding minimal latency, and providing evidence of where the defence itself is incomplete.*

2.2 Why GraphQL Makes BOLA Harder

In a REST API an object usually appears once, in the URL path (for example `/users/101/orders/501`), so one check per endpoint is often enough. GraphQL breaks this assumption in several ways:

1. **Dynamic selection sets and unbounded depth.** Clients build arbitrary nested queries with fragments, aliases and directives.
2. **Independent resolvers.** Each field has its own resolver. Checking the parent object does not protect the children.
3. **Implicit relationship traversal.** An attacker can reach a foreign object through a graph edge (for example `folder → children`) without ever writing its ID in the query.
4. **Mutations without ownership checks.** `update`, `delete` and `transfer` mutations can change other users' data unless they are intercepted before the resolver runs.
5. **Perimeter blindness.** Web Application Firewalls inspect HTTP traffic but do not understand the GraphQL schema, so they cannot decide who owns the objects a query will reach.

Table 2.1 lists the five forms of BOLA in GraphQL that this project handles.

Table 2.1 Forms of BOLA in GraphQL

Form	Example	Target ID in query?
Direct	`resource(id: 102)`	Yes
Nested	`user(id:101){resource(id:102){name}}`	Yes, below the root
Syntactic variant	variable, variable default, alias, fragment, directive, batch	Only after coercion
Relationship	`resource(id:201){children{id name}}`	**No**
Write	`updateResource(id: 102, name: "Hacked")`	Yes, as an argument

2.3 Motivation

- BOLA is the most common and most damaging API vulnerability class and directly leads to data breaches of personal, medical and financial records.
- Existing tools either *test* for BOLA before deployment or rely on developers writing a correct permission rule for every field. A single forgotten rule silently leaks data.
- Pre-execution checks cannot see relationship traversal; per-resolver checks see everything but are expensive to write and maintain. Neither tells the developer *where* it is incomplete.
- In GraphQL, a wrongly placed denial can trigger *null propagation*: an error in a non-null field nulls its nearest nullable parent [13], removing the user's own legitimate data and hurting availability.

A defence that is both complete (catches every form) and self-measuring (reports its own gaps) is therefore valuable for real network-facing services.

CHAPTER 3

OBJECTIVES

The objectives of this project are:

1. **Block every form of BOLA** in GraphQL – direct, nested, syntactic variants, relationship traversal and unauthorised writes – for authenticated non-administrator users.
2. **Defence in depth:** build two independent enforcement layers (pre-execution and execution-time) so that one layer's blind spot is covered by the other.
3. **Cross-layer coverage evidence:** make Layer 2 record every object Layer 1 never evaluated, turning defence in depth into a measurement of Layer 1's coverage.
4. **Blind-spot prediction:** train an XGBoost model, labelled automatically from that evidence, to predict which requests will reach objects Layer 1 cannot see, with SHAP explanations.
5. **Self-repairing configuration:** convert the evidence into configuration repairs, verify them by offline replay, and apply them only with administrator approval.
6. **Formal security properties:** state Safety, Non-interference, Write safety and Availability, and check them with property-based testing.
7. **Close side channels:** return uniform `ACCESS_DENIED` responses and equal timing for foreign and missing objects, so attackers cannot enumerate objects or owners.
8. **Secure accounts:** protect the authentication layer with salted PBKDF2 password hashing, login rate-limiting, account lockout and token revocation.
9. **Portability and low overhead:** support several Python GraphQL frameworks and keep the added latency to about 1 ms per request.
10. **Rigorous evaluation:** layer ablation, fault injection, comparison with graphql-shield, latency and scaling benchmarks.

CHAPTER 4

LITERATURE SURVEY

We reviewed work on BOLA detection, access-control enforcement, GraphQL analysis and the security and testing foundations this project builds on. The most relevant studies are discussed below and summarised in Table 4.1.

4.1 BOLA Detection by Testing

GraphQLer (Tsai et al., 2025) [2] builds a dependency graph from schema introspection, chains queries and mutations, and replays requests as other users to find authorization flaws. AuthProbe (Barach, 2026) [3] tests REST APIs from an OpenAPI specification with multiple identities, and Santos Filho et al. (2025) [4] transform OpenAPI specifications into Coloured Petri Nets to derive BOLA attack models. All three are test-time tools, and two of them target REST; they find flaws before deployment but do not block attacks at runtime.

4.2 BOLA Enforcement and Analysis

BolaZ (Wu et al., 2026) [5] combines CodeQL taint tracking with runtime authorisation checks in Spring Boot services. It shares our view that static analysis needs a runtime backstop, but it targets Java REST services and does not use one layer to measure the other. Lambers et al. (2026) [6] define taint analysis for graph APIs aimed at broken access control, a static complement to runtime checks.

4.3 GraphQL Authorisation and Query Analysis

The graphql-shield library [7] attaches permission rules to types and fields as resolver middleware; it is the per-field approach we compare against. Hartig and Pérez (2018) [8] give GraphQL a formal semantics, Cha et al. (2020) [9] analyse query cost statically, and Wittern et al. (2019) [10] study real schemas empirically. These works bound how expensive a query is, not whose objects it reaches. Kaur (2026) [11] catalogues over 100 bug-bounty BOLA disclosures, including GraphQL global-identifier leakage, and CVE-2022-39275 in Saleor [12] shows that type-confused global IDs are a real attack.

4.4 Security and Testing Foundations

Layer 2 follows the reference monitor concept of Anderson (1972) [14] and the complete mediation and fail-safe defaults principles of Saltzer and Schroeder (1975) [15]. Our non-interference property follows Goguen and Meseguer (1982) [16]. Comparing two layers' decisions is a form of differential testing [17], planting bugs to judge a detector follows mutation testing [18], and properties are checked as in QuickCheck [19] using Hypothesis [20]. Turning automatically produced records into labels resembles weak supervision (Snorkel) [21], and the blind-spot predictor uses XGBoost [22] with SHAP explanations [23].

Table 4.1 Summary of Literature Survey

S.No	Work	Approach	Limitation for this project
1	OWASP API Top 10 (2023) [1]	Ranks BOLA as API1; describes causes and prevention.	Guidance only; no enforcement mechanism.
2	Tsai et al., GraphQLer (2025) [2]	Dependency graph from introspection; replays requests as other users.	Test-time tool; does not block attacks at runtime.
3	Barach, AuthProbe (2026) [3]	Specification-driven multi-identity BOLA testing from OpenAPI.	REST only; test-time.
4	Santos Filho et al. (2025) [4]	OpenAPI to Coloured Petri Nets to derive BOLA attack models.	REST only; detection, not enforcement.
5	Wu et al., BolaZ (2026) [5]	CodeQL taint tracking plus runtime checks in Spring Boot.	Java REST; no cross-layer measurement.
6	Lambers et al. (2026) [6]	Static taint analysis for graph APIs.	Static only; no runtime guarantee.
7	graphql-shield library [7]	Permission rules on types and fields as middleware.	Many rules; missing mutation rule allows the write; denial can null own data.
8	Hartig & Pérez (2018) [8]	Formal semantics and complexity of GraphQL.	Foundation; not about authorisation.
9	Cha et al. (2020) [9]	Static query cost analysis.	Bounds cost, not *whose* objects are reached.
10	Wittern et al. (2019) [10]	Empirical study of GraphQL schemas.	Descriptive only.
11	Kaur (2026) [11]	Taxonomy of 100+ bug-bounty BOLA disclosures.	Empirical; no defence.
12	CVE-2022-39275 (Saleor) [12]	Mutations did not validate the type in Relay global IDs.	Shows type-confusion BOLA is real.

S.No	Work	Approach	Limitation for this project
13	Anderson; Saltzer & Schroeder [14], [15]	Reference monitor; complete mediation; fail-safe defaults.	Principles that Layer 2 implements.
14	Goguen & Meseguer (1982) [16]	Non-interference security model.	Basis for property P2.
15	McKeeman; Jia & Harman [17], [18]	Differential testing; mutation testing.	Basis for layer comparison and fault injection.
16	QuickCheck; Hypothesis [19], [20]	Property-based testing.	Used to check P1–P4.
17	XGBoost; SHAP [22], [23]	Gradient boosting; feature explanations.	Used for the blind-spot predictor.
18	Snorkel (2017) [21]	Weak supervision from automatic labels.	Our evidence labels requests similarly.

4.5 Key Observations

- **Testing tools** (GraphQLer, AuthProbe, Petri-net approaches) find BOLA before deployment but provide no runtime protection, and most target REST.
- **Enforcement tools** either analyse the query statically (and miss relationship traversal) or require a hand-written rule per field (and leak when one is forgotten).
- **GraphQL research** focuses on semantics and query cost, not object ownership.
- **Security foundations** – reference monitors, complete mediation, non-interference – are well known but have not been combined into a measurable, self-checking GraphQL defence.

CHAPTER 5

RESEARCH GAP

From the literature survey we identified the following gaps:

1. **No runtime defence that covers every GraphQL BOLA form.** Pre-execution analysis cannot see objects reached through relationships; per-resolver checks are written field by field and fail silently when one is missing.
2. **No self-measurement.** We found no prior work that enforces BOLA in GraphQL at two layers *and* uses one layer to measure the other's coverage. Existing defences cannot tell the developer where they are incomplete.
3. **No automatic, verified repair.** When an authorisation configuration is wrong, existing tools neither diagnose which entry is wrong nor propose a fix that is checked before it is applied.
4. **Availability is ignored.** Rule-based GraphQL authorisation can null an entire parent object because of GraphQL null propagation, deleting the user's own data from the response.
5. **Side channels are not addressed.** Different error messages or response times for “foreign” and “missing” objects let an attacker enumerate which objects exist and who owns them.
6. **No stated, tested security properties.** Existing GraphQL authorisation tools do not define safety or non-interference properties and test them systematically.
7. **Limited portability.** Solutions are tied to one framework or language.

GraphQL-Shield is designed to close each of these gaps.

CHAPTER 6

PROPOSED SYSTEM

GraphQL-Shield is a dual-layer runtime object guard for GraphQL APIs. It is configured once per application by a small `ShieldConfig` (the schema, which object types are owned, and how to look up an object's owner), and it then protects the whole API without any authorisation code inside the resolvers.

6.1 Threat Model

The attacker is an **authenticated user** who wants to read or change objects owned by someone else. They hold a valid token for their own account, can send any syntactically valid request, can guess other users' identifiers, and can read the schema. Authentication is assumed sound and ownership comes only from the data store, never from client input. An owned object is accessible only to its owner and to administrators; owning a parent folder grants nothing about its children.

6.2 Security Properties

For a non-administrator caller *u* and any request *q*:

P1 Safety: The response to *q* contains no object *u* does not own.

P2 Non-interference: Changing only objects *u* does not own (their content, existence, or children placed under *u*'s folders) does not change the response to *q*.

P3 Write safety: *q* changes the owner or content of no object *u* does not own, and deletes none.

P4 Availability: If *q* names only objects *u* owns, they are returned.

6.3 Main Components

1. **Layer 1 – Schema-aware pre-execution gate.** Resolves every ID the request contains (literals, variables, variable defaults, lists, fragments, aliases, batches, Relay global IDs), looks up its owner, and rejects the whole request with HTTP 403 on the first foreign ID, so a write never starts.
2. **Layer 2 – Execution-time guard.** A Strawberry extension / graphql-core middleware. Checks mutation targets after the engine coerces arguments; checks every returned object

of an owned type (including behind interfaces and unions); removes foreign list items *before* non-null rules apply; answers aggregate fields such as `childrenCount` from visible objects only.

3. **Cross-layer evidence.** Every object Layer 2 checks that Layer 1 never evaluated is logged; if Layer 2 also denies it, an `L1_BLIND_SPOT_CAUGHT_BY_L2` event is recorded.
4. **Blind-spot predictor.** An XGBoost model on 17 pre-execution query features, trained on labels produced by the evidence. Advisory only – it never changes a decision.
5. **Self-repairing configuration.** Diagnoses the cause of each gap, proposes a configuration repair, verifies it by replaying recorded requests offline, and applies it only with administrator approval.
6. **Uniform denials and timing.** Every denial says `ACCESS_DENIED`; missing objects are checked and audited exactly like foreign ones.
7. **Account protection and audit.** PBKDF2-SHA256 passwords, login lockout, token revocation, role read from the database on every request, and a hash-chained audit log.
8. **Security dashboard.** Live audit data, an attack test console, layer events and switches, a per-request taint tracer, the ML models, self-repair and the research results (Section 8.9).

6.4 Advantages over Existing Approaches

- Covers all five BOLA forms, including relationship traversal that pre-execution checks cannot see.
- Much less configuration than per-field rules (2 owned types, 1 mutation entry, 1 aggregate and an owner lookup, against 8 permission entries and 4 rule functions in graphql-shield).
- Fails closed: a mutation with no known target is denied.
- Keeps the owner's own data even when a foreign child is removed (no null-propagation loss).
- Measures and repairs its own coverage gaps.

CHAPTER 7

SYSTEM ARCHITECTURE / METHODOLOGY

7.1 Overall Architecture

Fig 7.1 shows the architecture: two independent checks around the GraphQL engine, a shared configuration, and the record of where the layers disagree.

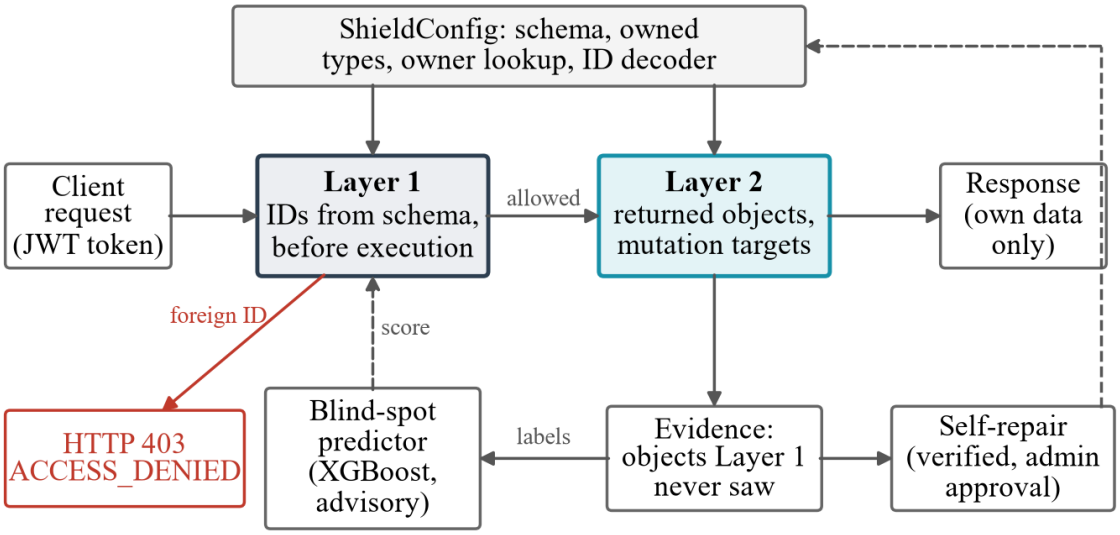


Fig 7.1 GraphQL-Shield Architecture

Layer 1 rejects requests naming foreign objects with HTTP 403; Layer 2 filters returned objects, blocks foreign writes and records every object Layer 1 never evaluated. That record feeds both the advisory blind-spot predictor and the self-repair module.

7.2 Request Lifecycle

Fig 7.2 shows how one request moves through the system, from authentication to the final response.

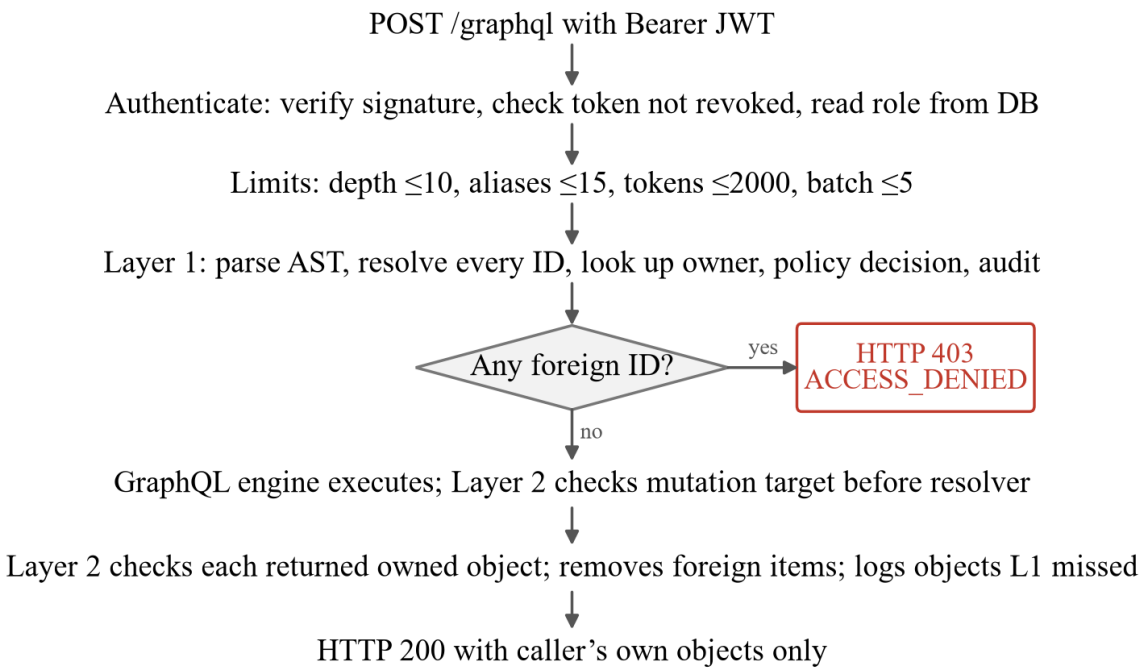


Fig 7.2 Lifecycle of a Request through GraphQL-Shield

7.3 Methodology

7.3.1 Per-Application Configuration

A `ShieldConfig` describes the protected application: its schema; which object types are owned, each mapped to an ownership key; an owner-lookup function (with an optional batched form); the arguments that carry identifiers and a decoder for them (for example Relay global IDs); targets for mutations whose return type does not reveal what they change; and aggregate fields. Everything else is derived from the schema. At startup a **lint** pass warns about unprotected types that have owner-like fields (`ownerId`, `createdBy`, …) and about root lookups by ID of types that are neither owned nor declared public.

7.3.2 Layer 1: Pre-Execution Gate

Layer 1 walks the parsed request against the schema. For every field with an ID argument it takes the object type from the schema (not from the field name), resolves the value as the engine will – literals, variables, declared defaults, lists, named and inline fragments, aliases, each operation of a batch – and checks it with the policy engine. A global ID whose embedded type differs from the expected type (type confusion, as in CVE-2022-39275) is checked as both types. One denial rejects the whole request.

7.3.3 Layer 2: Execution-Time Guard

Before a root mutation runs, Layer 2 checks the argument the engine actually coerced; a denial raises an error and the resolver never executes. After each resolver returns, Layer 2 decides from the field's GraphQL return type whether the value is an owned object, resolving interfaces and unions to their concrete type. A denied single object becomes `null`; a denied list item is removed before non-null types are enforced. Lists are checked with one batched owner lookup and one audit transaction.

7.3.4 Cross-Layer Evidence

Layer 1 marks every (type, ID) it evaluated. Layer 2 writes an audit record only for objects Layer 1 never marked, so each record is direct evidence of a Layer 1 coverage gap. The audit log is hash-chained, so a deleted or edited record is detectable.

7.3.5 Blind-Spot Predictor

For each request Layer 1 allows, 17 structural features are computed before execution (depth, field count, relationship fields, ID arguments and how many Layer 1 resolved, variables, defaults, fragments, aliases, directives, batch size, mutation flag, caller role). An XGBoost classifier (120 trees, depth 4) predicts whether the request will reach objects Layer 1 cannot see, with SHAP explanations. It is advisory only.

7.3.6 Self-Repairing Configuration

Each gap gets a cause (Table 7.1). A repair is replayed offline through Layer 1 on recorded requests and is offered only if it closes gaps, blocks no legitimate request, has at least 90% of its new IDs recognised by the owner lookup, and does not reuse another owned type's ownership key. Only an administrator can apply it; it can be reverted.

Table 7.1 Gap Causes and the Repairs Proposed

Cause	Repair
ID came in an argument the configuration does not list	Add it to `id_arguments`
A mutation has no known target	Add a `mutation_targets` entry
A per-user type is not protected (from the lint)	Add it to `owned_types`
Relationship traversal (`children`)	None – Layer 2 covers it by design
Listed argument still missed by Layer 1	None – reported as a code bug for a developer

7.3.7 Behavioural Model (Advisory)

A second, separate model watches user behaviour instead of query structure. For each user's recent activity it computes ten session features – requests per window, distinct objects, requests per minute, repeated targets, recent denials, cross-owner attempts, nested relationship queries, maximum query depth, enumeration score and burst rate – and scores them with an XGBoost classifier and a deterministic rule. On 1,000 held-out sessions from a query family not seen in training (500 normal, 500 suspicious), XGBoost reached 99.2% accuracy (no false positives, 8 false negatives) and the rule baseline reached 100%. We therefore make no claim that the machine-learning model is better than the rule. Like the blind-spot predictor, it is advisory and never changes an allow or deny decision.

7.3.8 Evaluation Plan

Six research questions are evaluated: layer ablation (RQ1), fault detection by the evidence (RQ2), properties and leak channels (RQ3), comparison with graphql-shield (RQ4), latency (RQ5), and self-repair (RQ6).

CHAPTER 8

IMPLEMENTATION

8.1 Technology Stack

GraphQL-Shield is written in Python and runs as a FastAPI service. Table 8.1 lists the languages, frameworks and libraries used in each part of the system.

Table 8.1 Technology Stack

Component	Technology
Language	Python 3.13
Web framework	FastAPI 0.143, Uvicorn
GraphQL	Strawberry GraphQL 0.333, graphql-core 3.3, Graphene, Ariadne
Authentication	PyJWT (HS256 JSON Web Tokens), PBKDF2-SHA256 password hashing
Database	SQLite with write-ahead logging and connection pooling
Machine learning	XGBoost 3.4, scikit-learn, NumPy, pandas, SHAP
Testing	pytest, Hypothesis (property-based testing)
Comparison baseline	Node.js, graphql-shield 7.6.5
Front-end	HTML/JavaScript security dashboard
DevOps	Docker, GitHub Actions CI

8.2 Module Structure

Each security mechanism lives in its own module under `app/security/`, separate from the protected application in `app/graphql/`. Table 8.2 maps each main file to its responsibility.

Table 8.2 Main Source Modules

File	Responsibility
`app/main.py`	FastAPI entry point, routers, layer and repair APIs
`app/security/enforcement.py`	Layer 1 pre-execution gate
`app/security/object_extractor.py`	AST traversal: fragments, variables, defaults, lists
`app/security/runtime_object_guard.py`	Layer 2 execution-time guard and middleware
`app/security/shield_config.py`	Per-application configuration and startup lint
`app/security/authorization.py`	Policy engine (owner / admin / deny)
`app/security/layers.py`	Layer state, layer events, blind-spot recorder
`app/security/audit.py`	Hash-chained audit log
`app/security/blind_spot_model.py`	XGBoost blind-spot predictor

File	Responsibility
app/security/repair.py	Self-repairing configuration
app/core/accounts.py	Lockout, token revocation, password policy
app/graphql/	Schema, queries, mutations, types
app/clinic/	Second, independent application (Relay IDs)
app/static/dashboard.html	Security dashboard (Section 8.9)
scripts/	Benchmarks, training, fault injection
tests/	427 unit, integration, property and security tests

8.3 Layer 1 Implementation

Layer 1 is a FastAPI dependency on the protected `/graphql` route. It reads the JSON body (failing closed on malformed input), enforces the batch limit, and evaluates each operation:

Listing 8.1 Core of the Layer 1 Decision Loop (simplified from enforcement.py)

```
normalized = normalize_query(query, variables, config=config)
for obj in normalized["objects"]:
    owner_id = owner_lookup(obj["type"], obj["id"])
    mark_layer1_checked(request, obj["type"], obj["id"])
    context = AuthorizationContext(user_id=user_id, role=role,
        operation=normalized["operation"], object_type=obj["type"],
        object_id=obj["id"], path=obj["path"], owner_id=owner_id)
    result = policy_engine.evaluate(context)
    log_decision(..., decision=result["decision"], layer="L1")
    if result["decision"] == "DENY":
        record_layer_event(user_id, "L1", EVENT_L1_BLOCKED, ...)
        raise HTTPException(status_code=403,
            detail={"layer": "L1_PRE_EXECUTION",
                    "reason": "ACCESS_DENIED"})
```

8.4 Layer 2 Implementation

Layer 2 has two entry points that share the same logic:

- `RuntimeObjectAuthorizationExtension` – a Strawberry schema extension;
- `OwnershipMiddleware` – a middleware for any graphql-core server, including Graphene and Ariadne.

In both forms it:

- checks the coerced target of each root mutation using the `MUTATION_TARGETS` registry; a mutation missing from the registry is denied;
- guards objects by GraphQL return type (`info.return_type`), resolving interfaces and unions;

- filters lists item by item using one batched owner lookup;
- computes declared aggregates (e.g. `childrenCount`) from visible objects only;
- records `L1_BLIND_SPOT_CAUGHT_BY_L2` events for objects Layer 1 never evaluated.

8.5 Account and Network-Level Protection

Object-level checks assume that the caller's identity is genuine, so the authentication layer is hardened as well (`app/core/accounts.py`). Table 8.3 lists each attack on accounts and tokens and the protection against it.

Table 8.3 Account Protection

Attack	Protection
Password guessing	5 failures per username in 15 min lock it for 15 min (HTTP 429); 20 failures from one IP block that address
Username enumeration	Same reply and same time for wrong password and unknown user (dummy hash check)
Token tampering	JWT signature check rejects modified user ID or role
Stale / stolen token	Every request checks the DB: account exists, token not logged out, not older than last password change; role read from DB
Weak passwords	Minimum 12 characters, no username, not common, enough distinct characters
Demo passwords in production	Server refuses to start with `ENVIRONMENT=production`
Query abuse	Depth 10, aliases 15, tokens 2000, batch 5; optional introspection off

8.6 Applications Protected

- **Demo application** (Strawberry, code-first): users, documents and folders with integer IDs. Four demo personas: three users and one administrator.
- **Clinic application** (schema-first SDL on plain graphql-core, dict resolvers, Relay global IDs): patients, records and prescriptions with independent owners. It shares no code with the demo app and its resolvers have no ownership checks; it is protected only by `CLINIC_CONFIG`.
- **Graphene and Ariadne** sample apps protected by the same middleware.

Each application also exposes an unprotected baseline endpoint (`/graphql-insecure`, off by default) used only for comparison.

8.7 API Endpoints

The service exposes the protected GraphQL endpoint together with REST endpoints for authentication, layer control, blind-spot scoring, repairs and auditing. Administrative endpoints require an admin token. Table 8.4 lists the main endpoints.

Table 8.4 Main API Endpoints

Endpoint	Method	Description
`/graphql`	POST	Protected GraphQL gateway (both layers)
`/graphql-insecure`	POST	Vulnerable baseline for comparison
`/api/v1/auth/login`	POST	Authentication and JWT issuance
`/api/v1/auth/logout(-all)`	POST	Revoke one / all sessions
`/api/v1/security/layers`	GET/POST	Inspect or toggle Layer 1 / Layer 2
`/api/v1/security/layer-ablation`	GET	Ablation results
`/api/v1/security/blind-spot-score`	POST	Score a query with SHAP explanation
`/api/v1/security/repairs/apply`	POST	Apply a verified repair (admin)
`/api/v1/audit-logs`	GET	Audit log (admin)
`/dashboard`	GET	Security dashboard
`/health`	GET	Health check

8.8 Live Demonstration Scenarios

For the live demonstration the database is seeded with three ordinary users (IDs 101–103) and one administrator (ID 104). User 101 owns resource 101 and folder 201; folder 201 also holds document 204, which belongs to user 102. Table 8.5 shows the requests user 101 sends and what the gateway does with each.

Table 8.5 Demonstration Scenarios (User 101 as Attacker)

Demo	Request	Outcome
Legitimate access	`resource(id: 101)`	200 OK
Direct BOLA	`resource(id: 102)`	403, Layer 1
Nested BOLA	`user(id:101){resource(id:102){name}}`	403, Layer 1
Unauthorised mutation	`updateResource(id:102, name:"Hacked")`	Blocked; DB unchanged
Relationship traversal	`resource(id:201){children{id} childrenCount}`	200; child 204 removed
Admin override	Admin reads `resource(id: 102)`	200, ADMIN_OVERRIDE

8.9 Security Dashboard

The dashboard (`/dashboard`) is a single HTML/JavaScript page that reads live data from the audit API. It lets a reviewer run every attack and see every result in the browser, without writing any requests by hand. Table 8.6 summarises its three tabs.

Table 8.6 Dashboard Tabs and What They Show

Tab	What it shows and allows
Security Testing	Live counts of allowed, blocked and high-risk requests; a test console that runs the seven attack scenarios against the protected app, the insecure baseline and the clinic app; a side-by-side comparison of protected and insecure responses; a one-click run of eight standard checks; a per-request taint-to-authorization trace; and administrator switches that turn each layer off to show what it stops.
Research & ML Evaluation	The behavioural model and an interactive session scorer; the blind-spot predictor with SHAP feature importance and a "score without running" box; the self-repair panel, where an administrator introduces a configuration mistake, analyses, applies a verified repair and reverts it; and the experiment results of Chapter 9.
Audit & Security Verification	The allowed/denied decision mix and decisions over time; the results of the last automated run of the property and leak-channel tests; runtime provenance records (object, owner, path, parent); and a searchable audit log with an integrity check of its hash chain.

CHAPTER 9

RESULTS AND DISCUSSION

All experiments run in-process on temporary seeded databases on a Windows laptop with Python 3.13. Deterministic experiments are reported as counts; randomised ones as mean and spread over seeds.

9.1 RQ1: Layer Ablation

Six attacks were run by one user against another user's objects under every combination of layers (Table 9.1).

Table 9.1 Layer Ablation: the Layer that Stopped Each Attack

Attack	Both	L1 only	L2 only	Neither
Direct	L1	L1	L2	leaked
Nested lookup	L1	L1	L2	leaked
Inline fragment	L1	L1	L2	leaked
Relationship	L2	**leaked**	L2	leaked
Batch	L1	L1	L2	leaked
Update mutation	L1	L1	L2	leaked

Layer 1 alone leaks relationship traversal because the child's ID never appears in the query. Layer 2 alone stops every attack but only at execution time. Together, most attacks are stopped before execution and the remaining relationship case is stopped by Layer 2. The clinic application passes all 14 of its security tests through configuration alone.

9.2 RQ2: Cross-Layer Evidence as a Fault Detector

Real discovery. On 4,000 generated requests, the evidence revealed that Layer 1 ignored variables' declared default values: `query($id: Int = 102)` sent without variables reached object 102 unchecked. All 585 such requests were coverage gaps and 263 were attacks stopped only by Layer 2. The hand-written relationship rule reached F1 0.639 (attack recall 22.8%); XGBoost reached F1 1.000. After fixing Layer 1, attacks reaching only Layer 2 fell from 401 to 138 (−66%).

Fault injection. 17 realistic bugs were patched into Layer 1 at load time and each was run on 5 seeds of 1,000 requests (Table 9.2).

Table 9.2 Fault Injection: 17 Layer 1 Faults, 5 Seeds of 1,000 Requests Each

Measure	Result
Faults detected by the evidence alone	**17 / 17** (every seed)
False alarms on unmodified Layer 1	**0**
Prediction of affected requests (mean F1)	
Relationship rule	0.661
Evidence rule	0.966
Decision tree	0.957
Logistic regression	0.968
XGBoost (structural features)	0.967
XGBoost (all features)	0.978
Localisation: broken construct ranked first (14 nameable faults)	
Lift statistic	0.929
Decision tree	0.957
Logistic regression	1.000
XGBoost	1.000

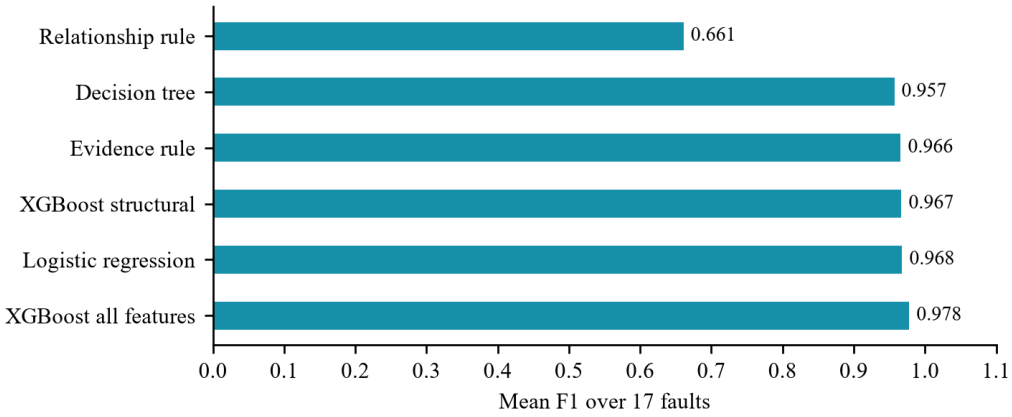


Fig 9.1 Predicting Requests Affected by Injected Layer 1 Faults

Every fault left unexplained gaps on every seed, and the unmodified layer left none. The simple evidence rule ("a relationship field, or an ID argument Layer 1 did not resolve") already reaches F1 0.966. XGBoost is no better than logistic regression: *the contribution is the evidence, not the model*.

9.3 RQ3: Security Properties and Leak Channels

P1–P4 were checked with Hypothesis (120 generated examples each). All four hold with protection on. With both layers switched off, P1, P2 and P3 fail, showing that the tests can detect

a violation; P4 (availability) cannot fail without protection, because nothing is blocked. Writing P2 exposed four leak channels, all now closed:

1. **Existence oracle:** different reasons for foreign and missing objects (and owner IDs in the explainer). Now always `ACCESS_DENIED`.
2. **Interfaces and unions:** `node(id:)` returned a foreign object's content. Layer 2 now resolves the concrete type.
3. **Aggregates:** `childrenCount` counted foreign children. Layer 2 now computes it from visible objects.
4. **Refused deletion:** deleting a folder with a hidden foreign child failed on a foreign-key constraint, revealing the child. Children now move to the top level.

Response timing was also measured: 1,500 interleaved pairs of requests for a foreign and a missing object were compared (Table 9.3).

Table 9.3 Timing Channel: Median Time of Foreign Minus Missing Object (ms)

Setting	Gap	95% CI	p
Read, both layers	+0.021	[−0.024, 0.060]	0.98
Update, both layers	+0.011	[−0.049, 0.065]	0.87
Read, Layer 2 only	−0.005	[−0.134, 0.122]	0.68
Update, Layer 2 only	+0.013	[−0.171, 0.223]	0.70
Read, Layer 2 only, *before fix*	+0.352	[0.193, 0.554]	2×10^{-8}

Before the fix, a foreign read took 0.352 ms longer than a missing one, a measurable timing side channel. After Layer 2 began checking missing lookups too, every confidence interval contains zero.

9.4 RQ4: Comparison with graphql-shield

The demo schema, data and unchecked resolvers were rebuilt in Node.js and protected with graphql-shield 7.6.5. Both ran the same 13 attacks and 5 benign scenarios, judged by one oracle (Table 9.4).

Table 9.4 Comparison with graphql-shield

Measure	GraphQL-Shield	graphql-shield
Attacks leaked	0 / 13	0 / 13
Legitimate requests failed	**0 / 5**	**1 / 5**
Configuration written	2 owned types, 1 mutation entry, 1 aggregate, owner lookup	8 permission entries, 4 rule functions
Missing mutation entry	0 leaks (derived or denied)	That write executes (1 leak)
Missing type entry	Type unprotected (9/13 or 1/13 leak)	0–1 leak (overlapping rules)
Deny fallback	–	0 leaks; 1–4 of 5 legitimate requests blocked

Both stop all attacks when fully configured. graphql-shield's failure is availability: in `children: [ResourceType!]!`, denying one foreign child nulls the whole folder, so the owner loses their own documents. GraphQL-Shield removes the item before nullability is enforced. Under configuration mistakes neither approach dominates: GraphQL-Shield fails closed on mutations but a missing owned type is a single point of failure (reported by the lint).

9.5 RQ5: Latency and Scaling

Fig 9.2 and Table 9.5 compare the protected and unprotected endpoints, measured with interleaved repeats so that machine drift affects every configuration alike.

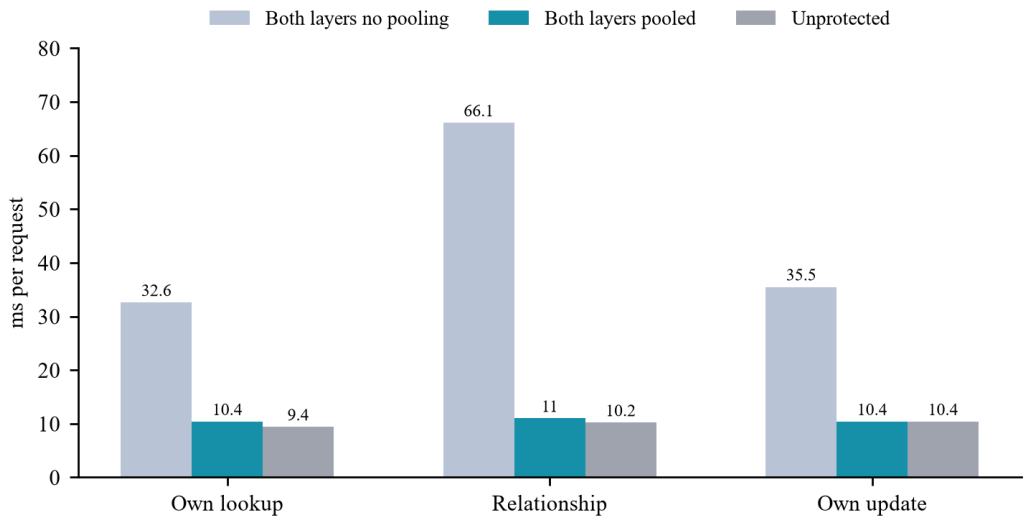


Fig 9.2 Mean Latency per Request (10 × 100 Interleaved Requests)

Table 9.5 Paired Overhead of Protection

Request	Unprotected	No pooling	Pooled	Overhead [95% CI]
Own lookup	9.4 ms	32.6 ms	10.4 ms	+0.99 ms [−0.36, +2.35]
Relationship	10.2 ms	66.1 ms	11.0 ms	+0.86 ms [+0.32, +1.40]
Own update	10.4 ms	35.5 ms	10.4 ms	+0.00 ms [−1.28, +1.29]

Table 9.6 Latency (ms) as a Folder Returns More Children (10% Foreign)

Children	Unprotected	Layer 2 one by one	Layer 2 batched
1	6.9	8.8	7.8
10	8.5	9.9	9.2
50	9.2	18.1	12.8
200	11.7	44.6	26.3

Protection costs about 1 ms per request (at most 2.4 ms). Three profiling-driven fixes achieved this: connection pooling (3.1–6.0× faster), an indexed existence probe in the risk engine instead of a full audit-log count, and single-threaded, cached blind-spot scoring with cached query parsing. Batched list checks matter as lists grow: at 200 children batching cuts the protected request from 44.6 to 26.3 ms (Table 9.6).

9.6 RQ6: Self-Repairing Configuration

Ten realistic configuration mistakes were planted across both applications; each ran three times (correct, mistaken, repaired) on 3 seeds of 400 requests (Table 9.7).

Table 9.7 Self-Repair: 10 Configuration Mistakes, 3 Seeds of 400 Requests

Measure	Result
Mistakes detected	**30 / 30**
First verified repair is the correct one	**30 / 30**
Responses restored exactly, in one round	**30 / 30**
Responses changed by a mistake	23–207 per run
Repairs proposed for correct configurations	**0** (6 runs)
Legitimate requests blocked by verified repairs	0
Wrong (decoy) repairs refused	**33 / 33**

Every mistake was detected and fixed by its first verified repair. In the first decoy run one wrong repair (protecting `UserType` with the `Resource` owner table) passed, because demo user IDs 101–

104 coincide with resources owned by the same users. This led to a new rule – a repair may not reuse another owned type's ownership key – after which all decoys were refused.

9.7 Test Suite

The project has **427 automated tests** (unit, integration, property-based and security tests); 423 pass in the main environment and 4 Graphene tests run in a separate environment because Graphene requires graphql-core < 3.3. Twenty-two of these tests cover account protection: the lockout after five failures, equal response time for unknown users, token revocation after logout and password change, roles read from the database, and the password policy.

9.8 Limitations

- Both applications, attacks, faults and configuration mistakes were written by the authors; results show the mechanism works, not that it generalises to third-party applications.
- Training data for the predictor is synthetic, so F1 close to 1.0 reflects an easy target, not real-world accuracy.
- A missing owned type is unprotected until the lint flags it and a repair is applied.
- Only owner-or-administrator policies are supported.
- Latency and timing were measured in-process on a laptop, without network delay.

CHAPTER 10

CONCLUSION

GraphQL-Shield shows that enforcing GraphQL object ownership at two independent points stops BOLA in every form we tested – direct, nested, syntactic variants, relationship traversal and unauthorised writes – and lets a gateway measure its own coverage. A pre-execution layer alone misses objects reached through relationships; an execution-time layer catches them and records each object the first layer never saw.

Planting 17 bugs in Layer 1 showed that this record detects every one with no false alarms, that a one-line rule over it predicts the affected requests (F1 0.966), and that simple models localise the broken construct as reliably as XGBoost. The same record repairs mistaken configurations: all 10 planted mistakes were restored by their first verified repair, and verification refused all 33 wrong repairs. Stating non-interference exposed and closed four leak channels, and no timing difference remains between foreign and missing objects.

The system needs only a small configuration, runs on four Python GraphQL frameworks, costs about 1 ms per request, and avoids the availability loss that per-field rules suffer under GraphQL null propagation. From a network-security perspective, the project demonstrates defence in depth, complete mediation, fail-safe defaults and side-channel resistance applied to a modern API protocol, together with a measurable way to know when the defence is incomplete.

CHAPTER 11

FUTURE SCOPE

Although the project met its objectives, several directions can extend it:

1. **Third-party evaluation:** evaluate on real open-source GraphQL applications (e.g. Saleor) and with independently written attacks, including black-box testers such as GraphQLer.
2. **Richer policies:** support sharing, teams, roles per object and attribute-based access control (ABAC) inside Layer 2's per-object check.
3. **Code-level repair:** extend self-repair from configuration to suggesting patches for Layer 1 code bugs.
4. **Better type discovery:** detect owned types reachable only through relationships, beyond the naming heuristic of the lint.
5. **Network-level timing analysis:** measure timing channels over real networks and add constant-time response padding if needed.
6. **Production deployment:** TLS termination, distributed caching (Redis) for the owner lookup, horizontal scaling, and integration with API gateways such as Kong or Envoy.
7. **Real traffic learning:** train the blind-spot and behavioural models on real anonymised traffic and add online learning.
8. **Other ecosystems:** port the middleware to Node.js (Apollo Server), Java (graphql-java) and Go (gqlgen).
9. **Stronger authentication:** add multi-factor authentication and OAuth 2.0 / OpenID Connect integration.
10. **SIEM integration:** stream the hash-chained audit log and layer events to Security Information and Event Management tools.

REFERENCES

[1] OWASP Foundation, "OWASP API Security Top 10 – 2023: API1:2023 Broken Object Level Authorization," 2023. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/

[2] O. Tsai, J. Li, T. T. Cheung, L. Huang, H. Zhu, J. Xiao, I. Sharafaldin, and M. A. Tayebi, "GraphQLer: Enhancing GraphQL security with context-aware API testing," arXiv:2504.13358, 2025.

[3] J. Barach, "AuthProbe: Specification-driven, multi-identity detection of broken object-level authorization in recruitment API," arXiv:2607.20574, 2026.

[4] A. Santos Filho, R. J. Rodríguez, and E. L. Feitosa, "Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored Petri nets transformation," *International Journal of Information Security*, vol. 24, no. 2, p. 83, 2025.

[5] A. Wu, Z. Feng, R. Feng, Z. Xing, and Y. Liu, "Rethinking broken object level authorization attacks under zero trust principle," *ACM Transactions on Software Engineering and Methodology*, 2026.

[6] L. Lambers, L. Sakizloglou, T. Khakharova, and F. Orejas, "Taint analysis for graph APIs focusing on broken access control," *Logical Methods in Computer Science*, vol. 22, no. 1, 2026.

[7] graphql-shield, "GraphQL server permissions as another layer of abstraction," npm package, version 7.6.5. [Online]. Available: https://www.npmjs.com/package/graphql-shield

[8] O. Hartig and J. Pérez, "Semantics and complexity of GraphQL," in *Proc. World Wide Web Conference (WWW)*, 2018, pp. 1155–1164.

[9] A. Cha, E. Wittern, G. Baudart, J. C. Davis, L. Mandel, and J. A. Laredo, "A principled approach to GraphQL query cost analysis," in *Proc. ESEC/FSE*, 2020, pp. 257–268.

[10] E. Wittern, A. Cha, J. C. Davis, G. Baudart, and L. Mandel, "An empirical study of GraphQL schemas," in *Service-Oriented Computing (ICSOC)*, LNCS, 2019, pp. 3–19.

[11] B. Kaur, "Broken object level authorization in the wild: An empirical taxonomy from 100+ bug bounty disclosures," arXiv:2605.25865, 2026.

[12] "CVE-2022-39275: Saleor mutations did not validate object ID types," 2022. [Online]. Available: https://db.gcve.eu/vuln/CVE-2022-39275

[13] GraphQL Foundation, "GraphQL specification: Handling field errors." [Online]. Available: https://spec.graphql.org/

[14] J. P. Anderson, “Computer security technology planning study,” U.S. Air Force Electronic Systems Division, Tech. Rep. ESD-TR-73-51, 1972.

[15] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[16] J. A. Goguen and J. Meseguer, “Security policies and security models,” in *Proc. IEEE Symposium on Security and Privacy*, 1982, p. 11.

[17] W. M. McKeeman, “Differential testing for software,” *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[18] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[19] K. Claessen and J. Hughes, “QuickCheck: A lightweight tool for random testing of Haskell programs,” in *Proc. ACM SIGPLAN ICFP*, 2000, pp. 268–279.

[20] D. R. MacIver, Z. Hatfield-Dodds, et al., “Hypothesis: A new approach to property-based testing,” *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[21] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, “Snorkel: Rapid training data creation with weak supervision,” *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.

[22] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.

[23] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

APPENDIX – RESEARCH PAPER TEMPLATE

The work in this report has been written up as a research paper in the **IEEE conference template** (two-column). It is provided as the separate file *paper/main.pdf* in the project folder, compiled from the LaTeX source `paper/main.tex` with references in `paper/references.bib`.

Table A.1 Research Paper Details

Item	Details
Title	GraphQL-Shield: Dual-Layer Enforcement Against GraphQL BOLA with Cross-Layer Coverage Evidence
Authors	[Author 1], [Author 2], [Author 3], [Author 4]
Template	IEEE Conference (two-column)
Keywords	GraphQL, API security, broken object level authorization, access control, defense in depth, fault injection, property-based testing
References	23 (IEEE citation style)
Figures / Tables	2 figures (architecture, latency), 7 tables

Structure of the Paper

I. **Introduction** – BOLA in GraphQL and five contributions.

II. **Background** – GraphQL execution, forms of BOLA, global IDs and type confusion, abstract types, null propagation.

III. **Related Work** – BOLA testing, BOLA enforcement, GraphQL authorisation, security foundations.

IV. **Threat Model and Properties** – attacker model; properties P1–P4.

V. **Design** – configuration and lint, Layer 1, Layer 2, uniform denials, cross-layer evidence and predictor, self-repair.

VI. **Implementation** – FastAPI, graphql-core, SQLite; demo and clinic applications; Graphene and Ariadne.

VII. **Evaluation** – RQ1 ablation, RQ2 fault injection, RQ3 properties and leak channels, RQ4 comparison, RQ5 latency, RQ6 self-repair.

VIII. **Discussion and Limitations** – external validity, synthetic traffic, configuration, residual channels, measurement.

IX. **Conclusion**